SMART E-COMMERCE PLATFORM

Project Documentation

Submitted by: Ram Kumar
Project: Smart E-Commerce Platform
Technology: Python, FastAPI, MySQL, JWT, Auth0, Postman
Repository: GitHub – Smart E-Commerce Platform

Githublink: https://github.com/RAMKUMAR63815/smart-ecommerce-platform

---

1. INTRODUCTION

The Smart E-Commerce Platform is a full-stack e-commerce application developed as part of a backend/full-stack development assignment.
The system provides secure authentication, product management, shopping cart, order processing, payment management, and administrative APIs.

2. PROJECT OBJECTIVE

The main objective is to develop a secure and modular e-commerce platform with REST APIs.
The application focuses on authentication, authorization, database management, and complete e-commerce workflows.

3. PROJECT SCOPE

The project covers user authentication, products, cart, orders, payments, dashboard, analytics, and administration.
Swagger and Postman are used for API documentation and testing.

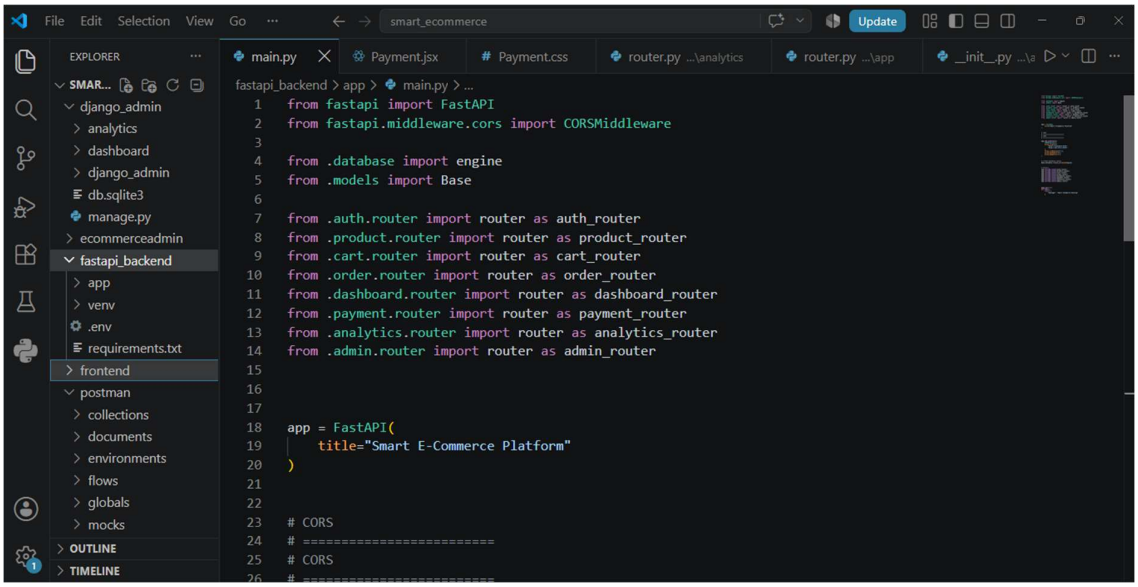
4. TECHNOLOGY STACK

Backend: Python, FastAPI, SQLAlchemy, Pydantic
Database: MySQL
Authentication: JWT, Auth0
Testing: Swagger UI, Postman
Version Control: Git, GitHub
Frontend/Admin: React/Next.js and Django

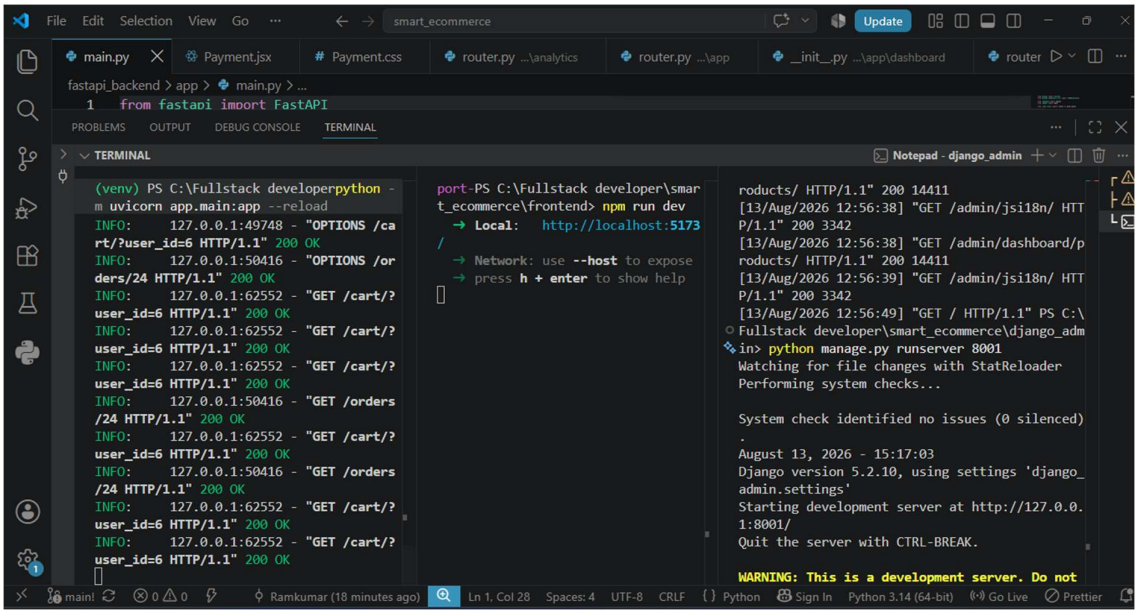
5. SYSTEM ARCHITECTURE

The application follows a modular architecture where backend APIs, database operations, authentication, administration, frontend, and testing are separated.
FastAPI manages the REST APIs and SQLAlchemy provides database communication with MySQL

5.1 SYSTEM ARCHITECTURE

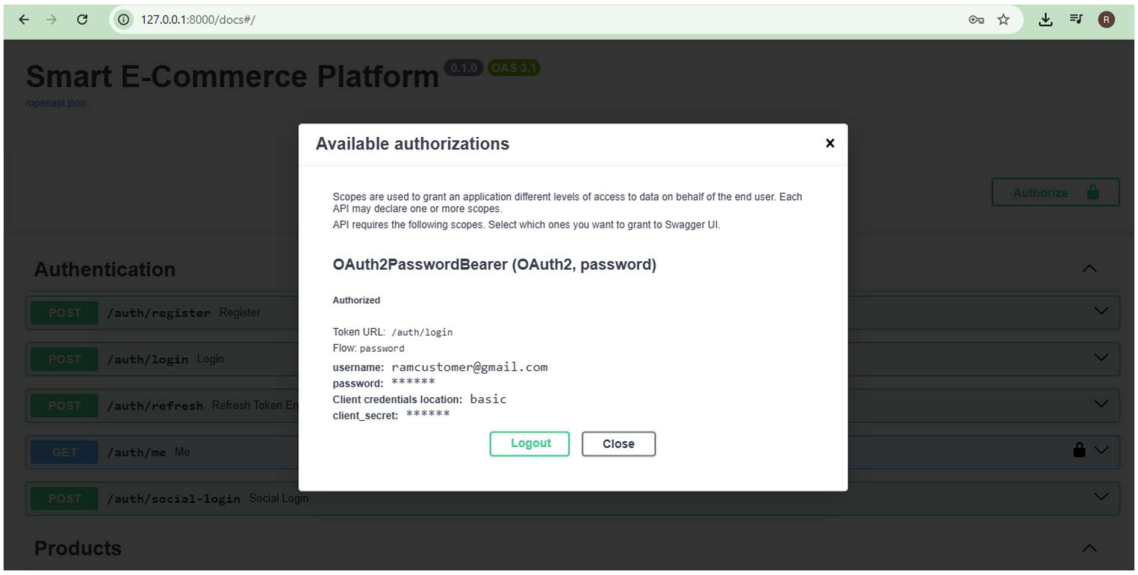


6. FRONTEND , BACKEND & DJANGO INITIALIZATION USING COMMANDS



7. USER AUTHENTICATION

Users can register and log in using their email and password.
Passwords are securely hashed and JWT tokens are generated after successful authentication.

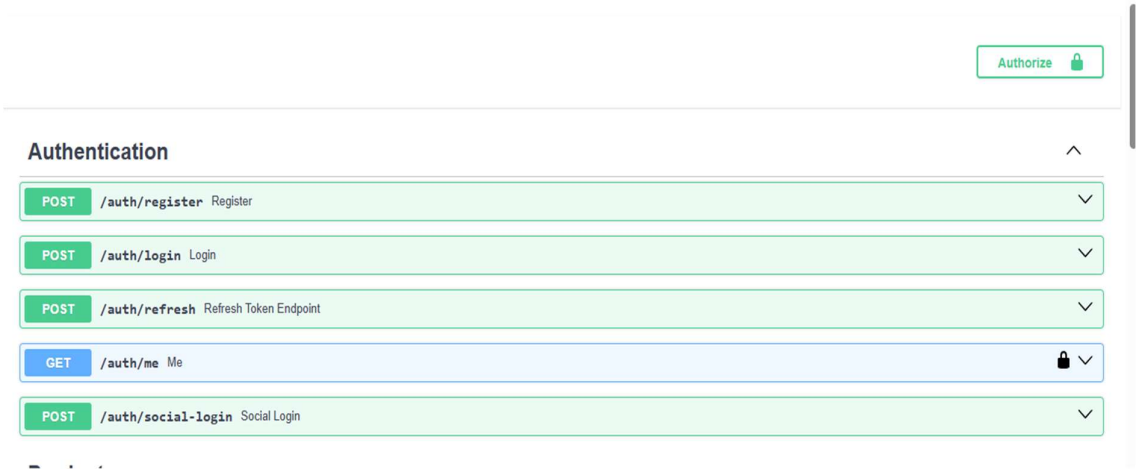


8. JWT AUTHENTICATION

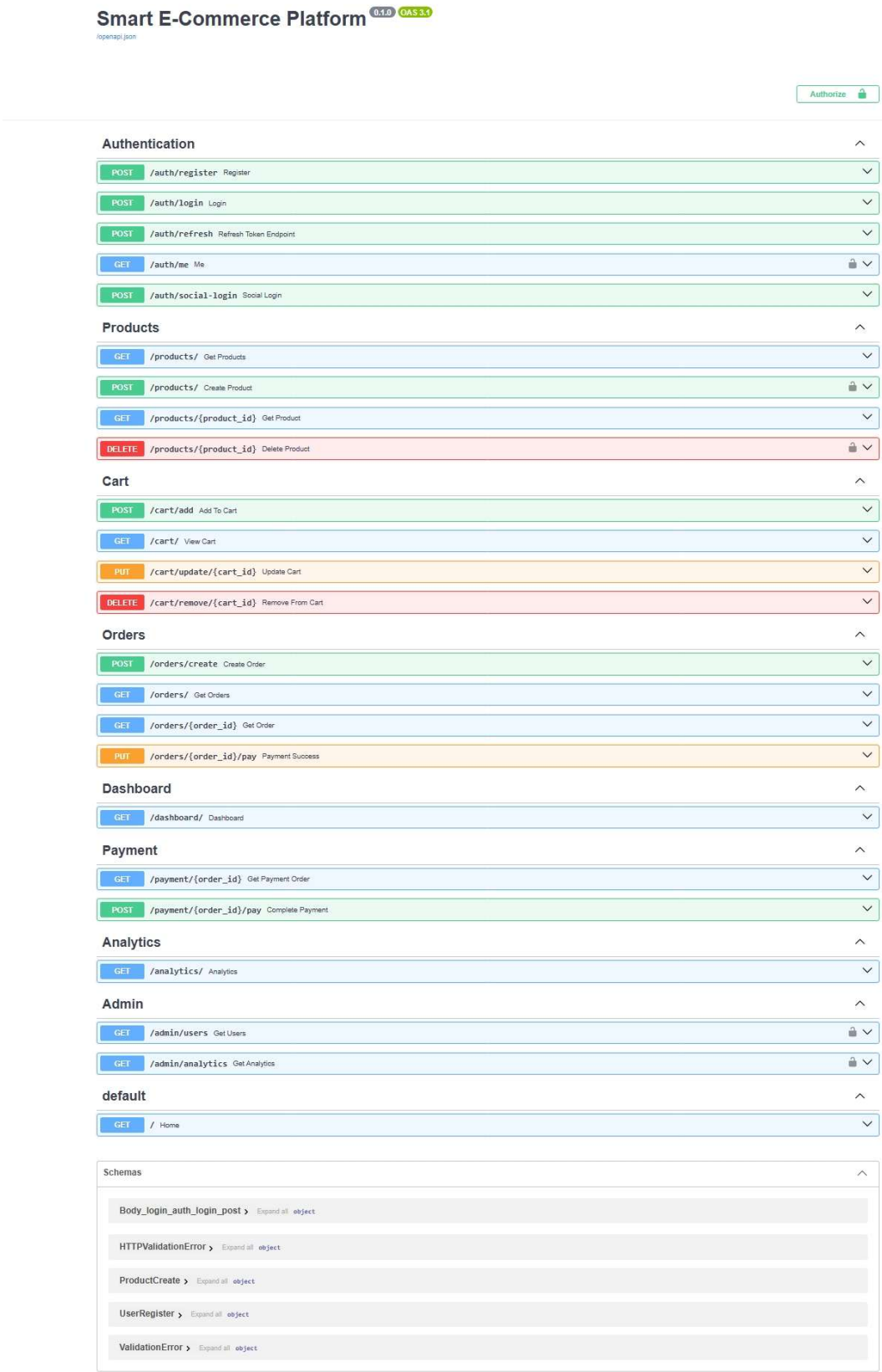
The system uses access tokens to authenticate requests to protected endpoints.
Refresh tokens allow users to obtain a new access token when the existing token expires.

9. SOCIAL LOGIN

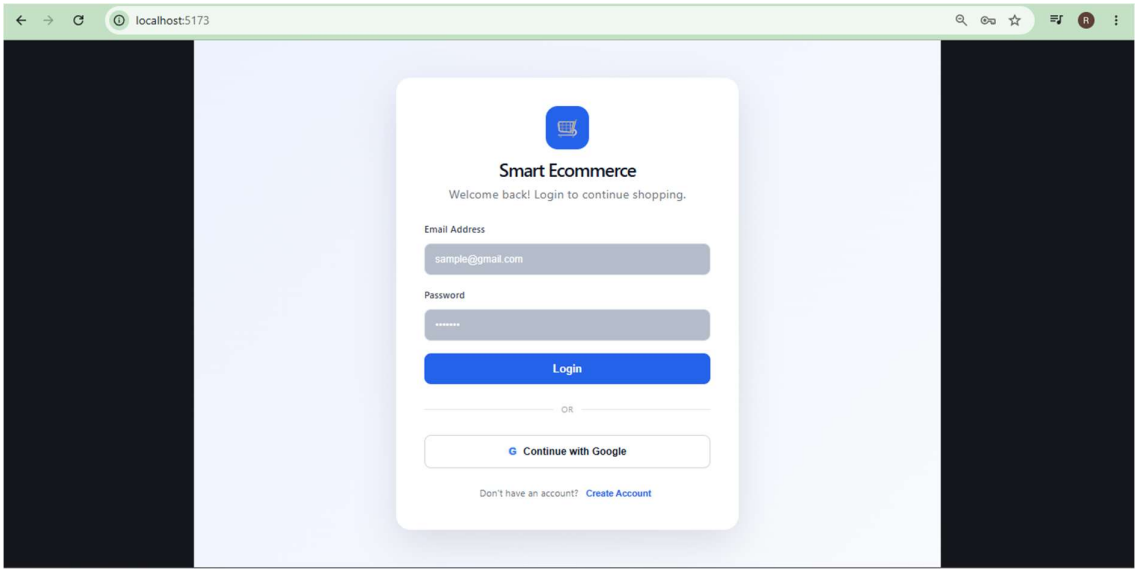
Auth0 is integrated to provide social authentication functionality.
Google and Facebook can be used as external authentication providers through Auth0.



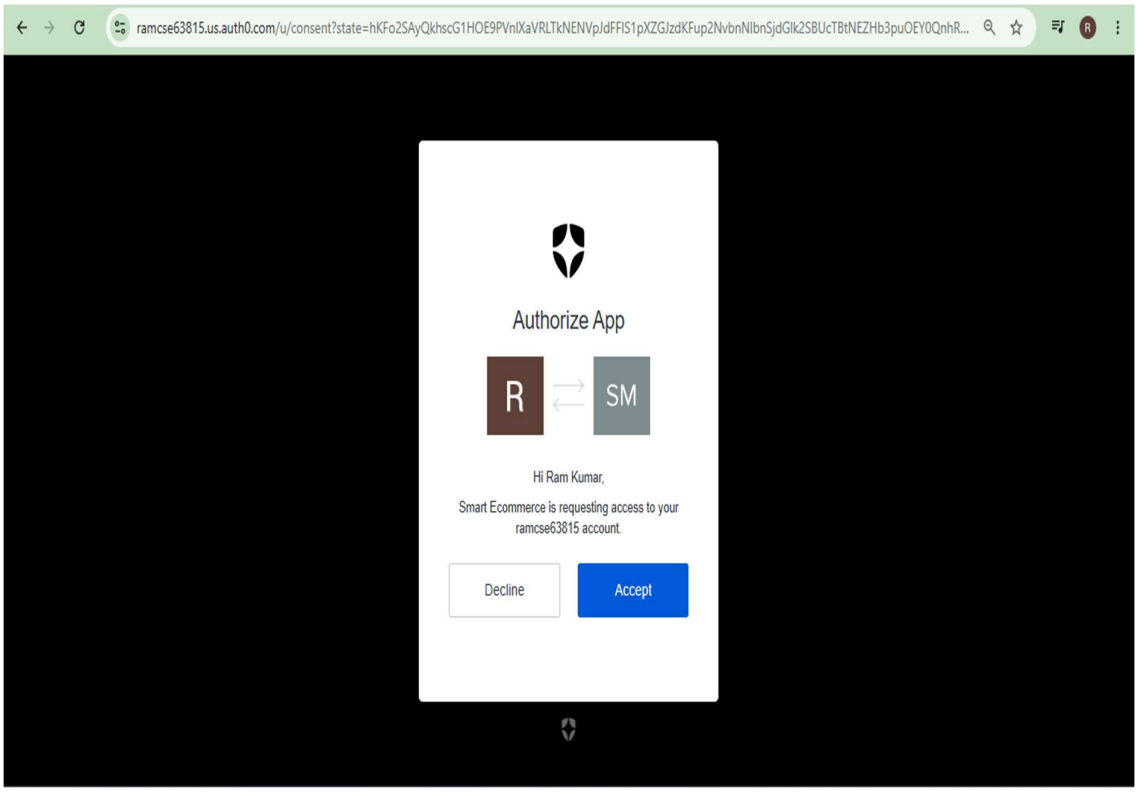
10. SWAGGER UI FULL PAGE:



11. FRONTEND LOGIN PAGE USING GENERAL LOGIN AFTER REGISTER:



12:lOGIN USING GOOGLE OR FACEBOOK LOGIN :



10. ROLE-BASED ACCESS CONTROL

The application supports Admin, Staff, and Customer roles.
Each role receives appropriate permissions for accessing protected application functionality.

11. PRODUCT MANAGEMENT

The Product module provides APIs for managing product information.
Authorized users can create, view, update, and delete products according to their assigned permissions.

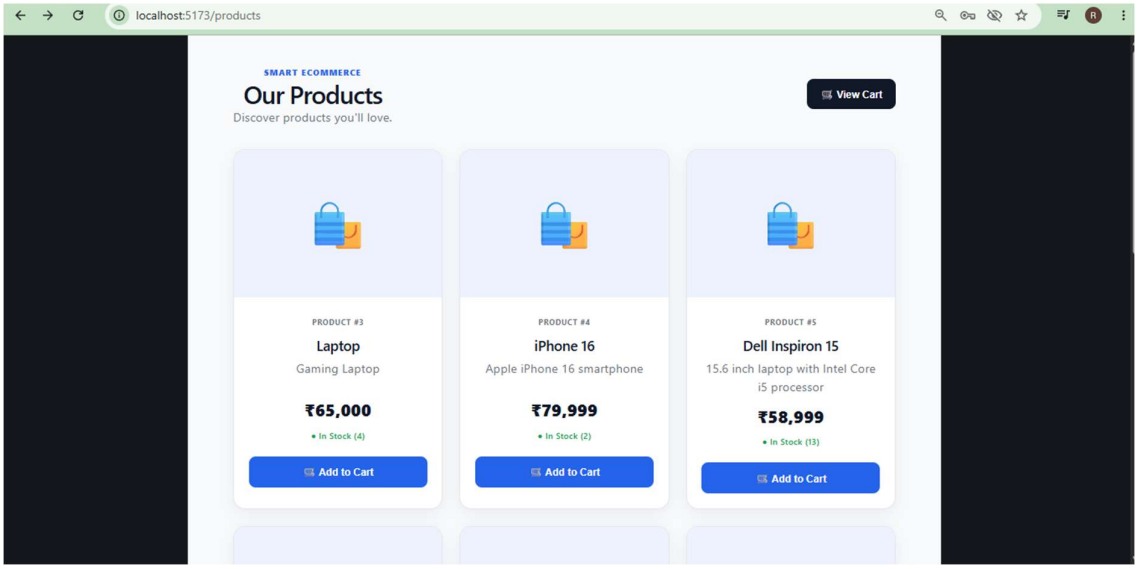


Fig11.1

Fig11.2

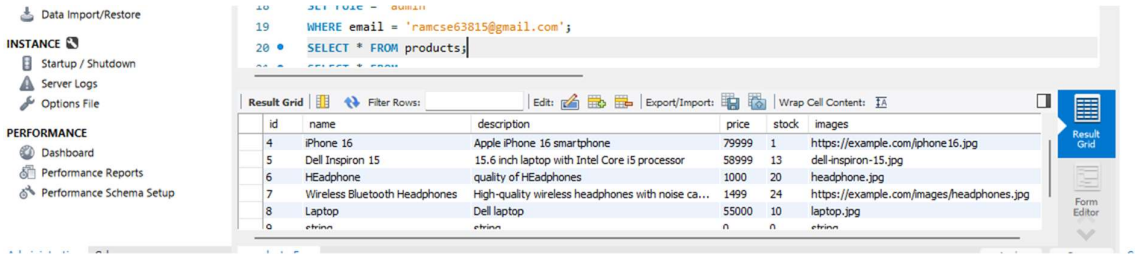
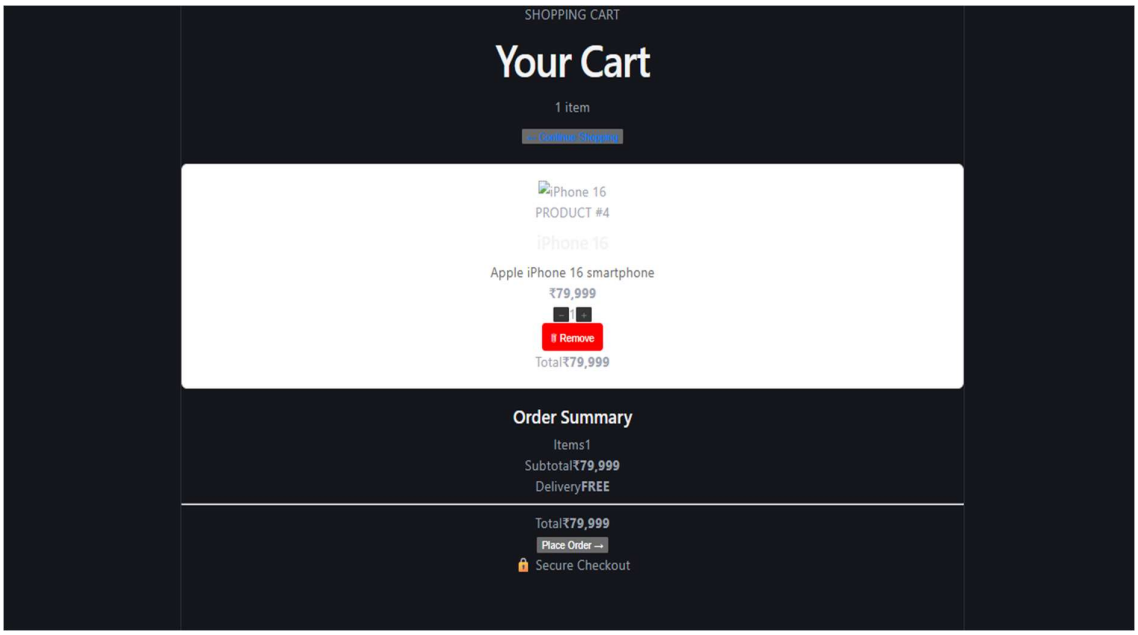


Fig11.3

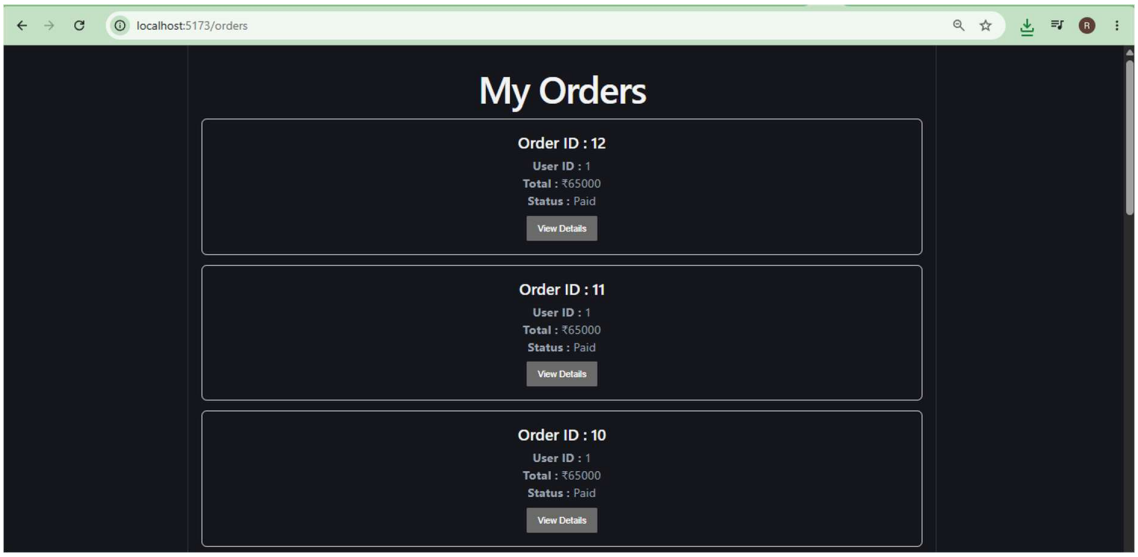
12. SHOPPING CART

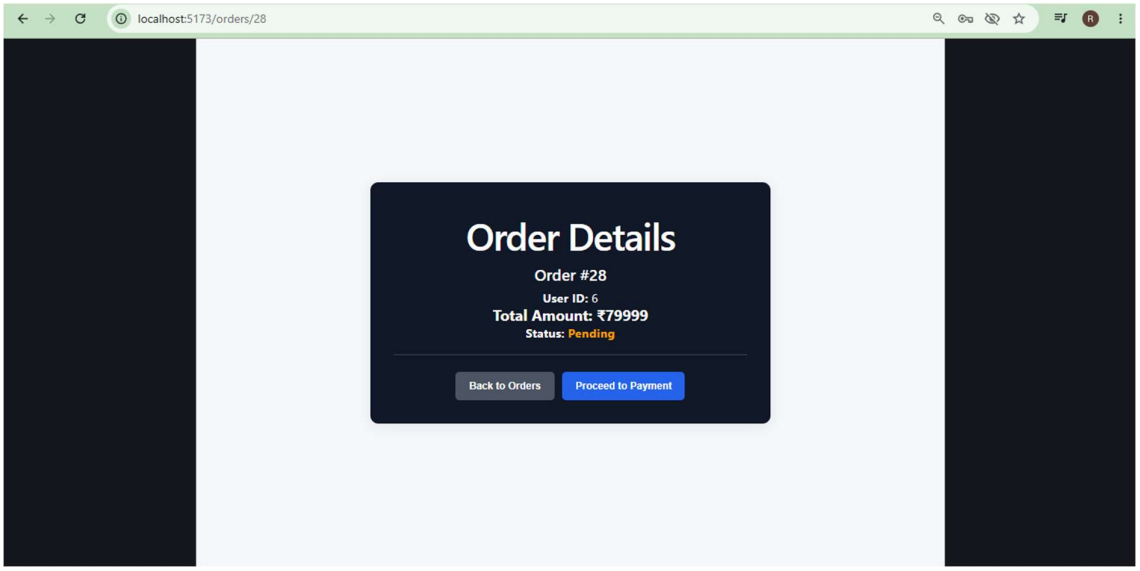
Customers can add products to their cart and manage product quantities.
The cart module provides APIs for viewing, updating, and removing cart items.



13. ORDER MANAGEMENT

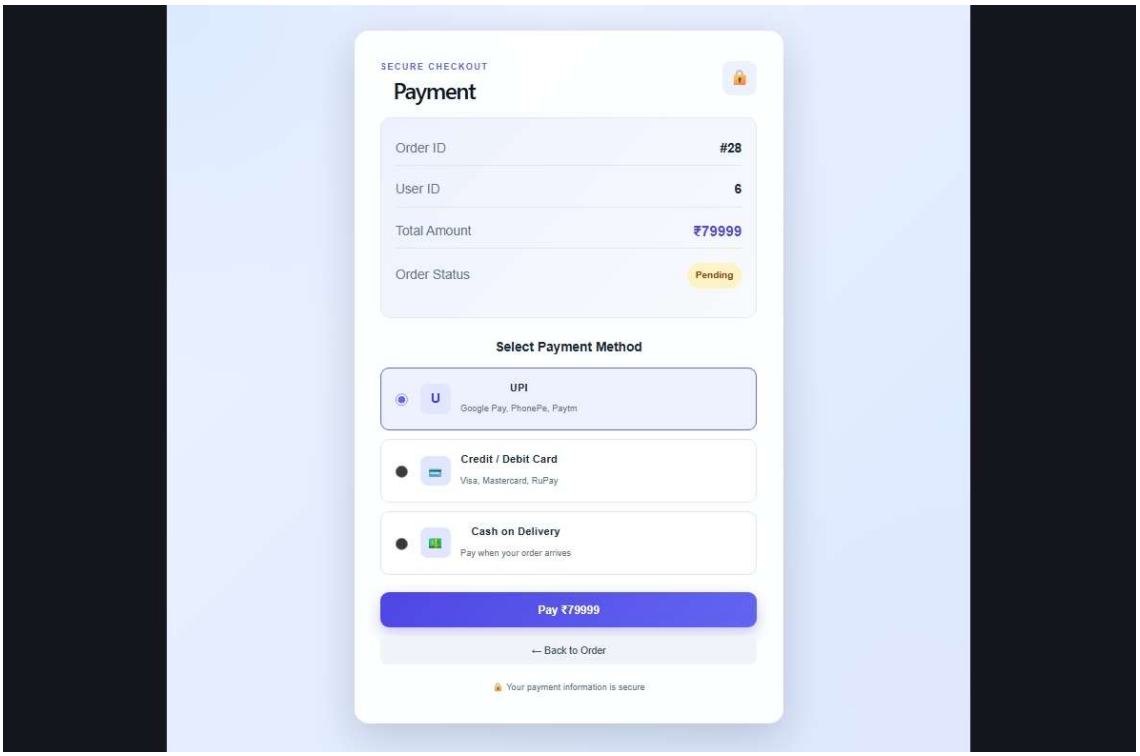
Customers can create orders from their selected products and view their order information.Order status is maintained throughout the purchasing workflow.

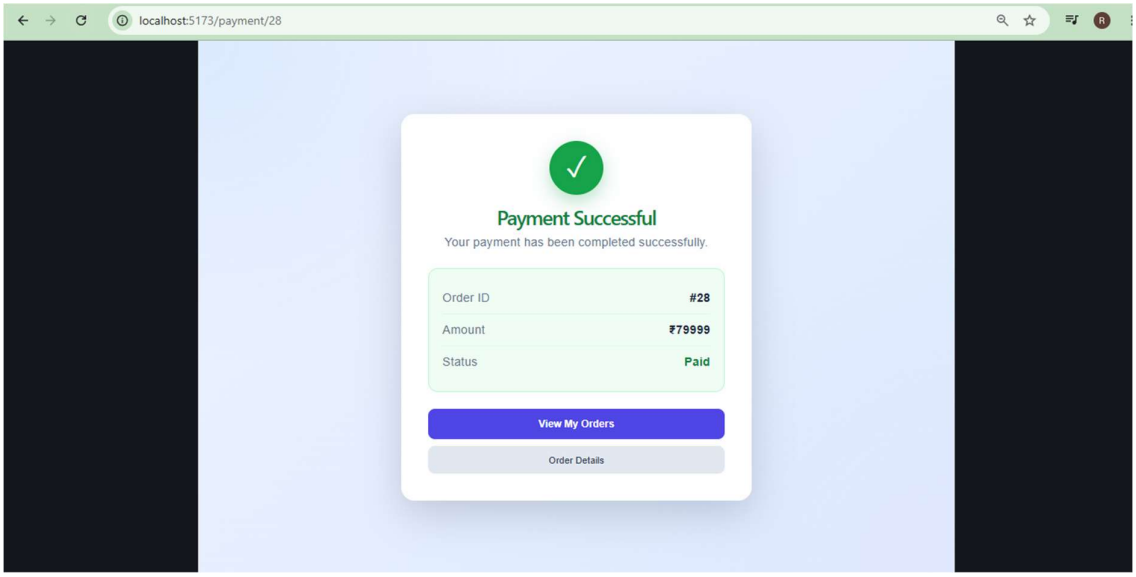




14. PAYMENT MANAGEMENT

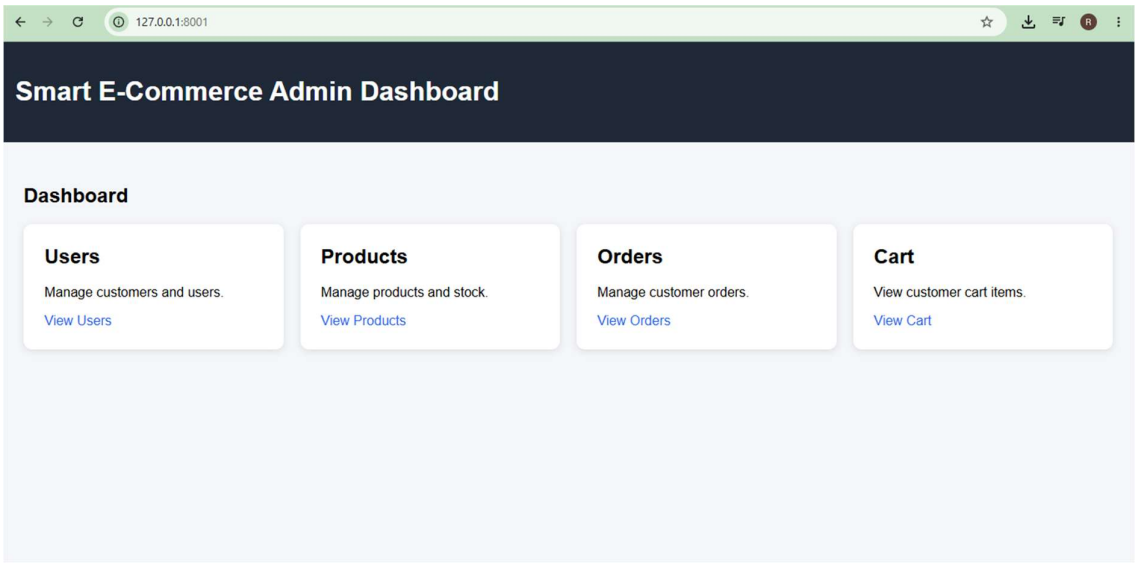
The Payment module manages payment-related information associated with orders. Payment operations update the corresponding order and payment status.





17. DJANGO ADMINISTRATION:

Administrative APIs provide authorized users with management capabilities.
Access to administrative functionality is protected using JWT authentication and role-based authorization

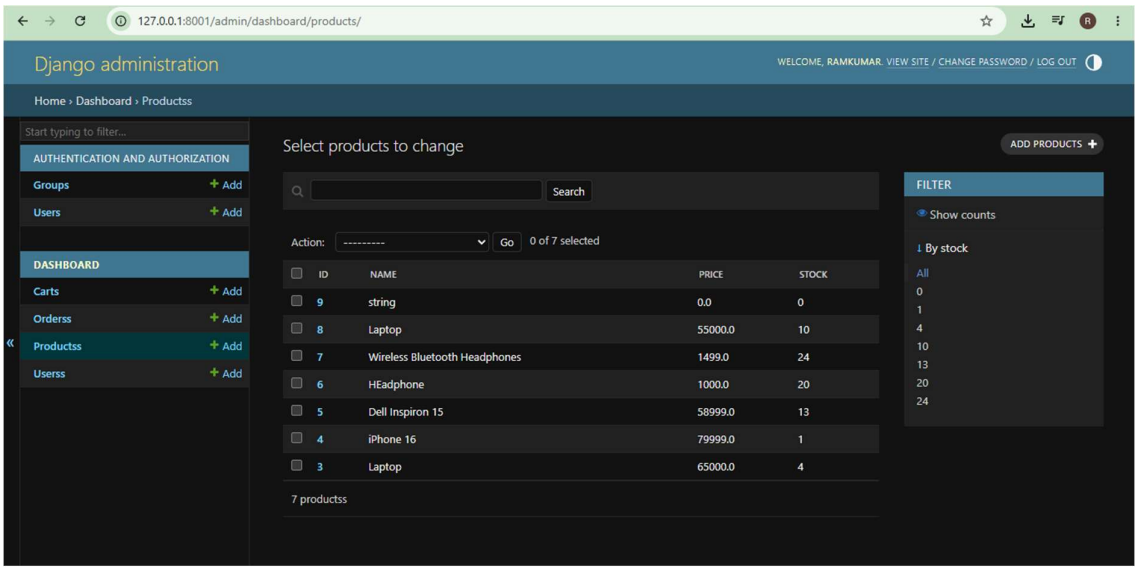


15. DASHBOARD

The Dashboard provides summarized information about the application.
It can display statistics such as users, products, cart items, and orders.

16. ANALYTICS

Analytics APIs provide useful application statistics for administrative monitoring.
These APIs can be extended to provide detailed sales, order, product, and user analytics.



Screenshot for Dashboard & Analytics

19. API DOCUMENTATION

FastAPI automatically generates Swagger/OpenAPI documentation.
The Swagger interface can be accessed through:

http://127.0.0.1:8000/docs

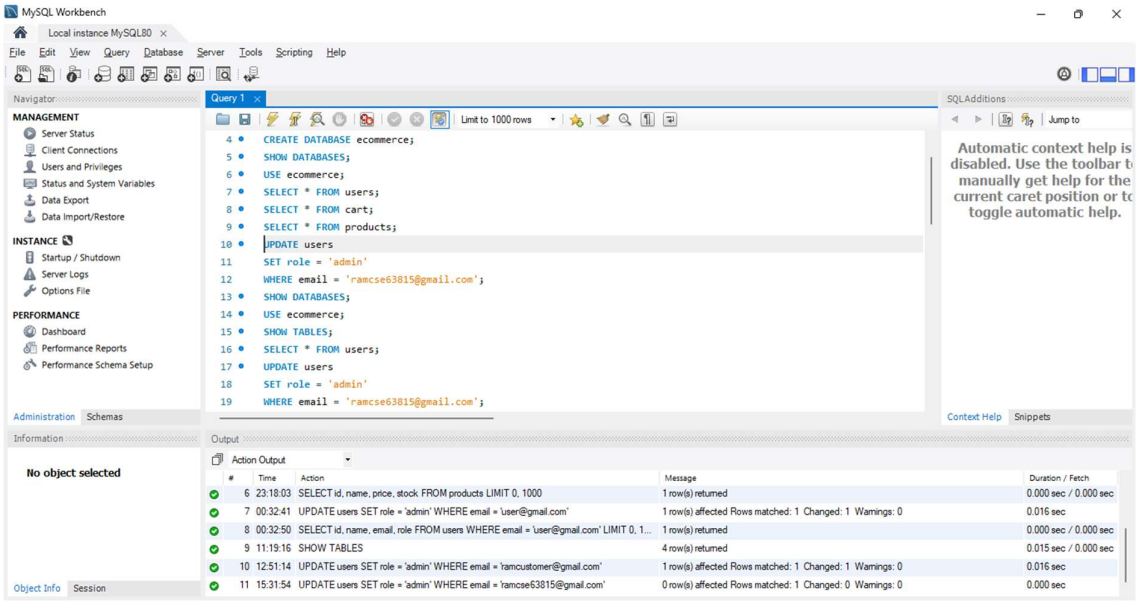
20. POSTMAN TESTING

A Postman collection has been created for testing the application's APIs.
The collection includes authentication, product, cart, order, payment, dashboard, analytics, and administrative requests.

20. DATABASE DESIGN

The database contains important entities such as Users, Products, Cart, Orders, and Payment information.

Relationships between these entities support the complete shopping and order-processing workflow.



21. TESTING WORKFLOW

The recommended testing process starts with user registration followed by login and JWT token generation.
The token is then used to test protected endpoints such as /auth/me, products, cart, orders, and administrative APIs.

22. SECURITY

The application implements password hashing, JWT authentication, token expiration, protected routes, and role-based authorization.
Sensitive database credentials, secret keys, and Auth0 credentials are stored using environment variables.

23. ENVIRONMENT CONFIGURATION

Environment variables are used for database configuration, JWT settings, and authentication credentials.
The real .env file is excluded from GitHub and an .env.example file is provided as a configuration reference.

24. ERROR HANDLING

FastAPI HTTP exceptions and validation mechanisms are used to handle invalid requests and unauthorized access.
The API returns appropriate HTTP status codes and error messages.

25. PROJECT STRUCTURE

```
smart_ecommerce/
│
├── fastapi_backend/
├── django_admin/
├── frontend/
├── postman/
├── .env.example
├── .gitignore
└── README.md
```

26. VERSION CONTROL

Git is used to maintain the source-code history and track project changes.
The completed project is hosted on GitHub for version control and assignment submission.

27. PROJECT DELIVERABLES

The completed assignment includes the FastAPI backend, database integration, authentication, authorization, product, cart, order, payment, dashboard, analytics, and Postman testing.
The project also contains documentation, environment configuration templates, Git configuration, and the GitHub repository.

28. FUTURE ENHANCEMENTS

Future improvements can include Razorpay/Stripe integration, product search, filtering, pagination, email notifications, cloud image storage, and automated testing.
Docker deployment, CI/CD pipelines, and cloud deployment can also be implemented.

29. GITHUB REPOSITORY

Repository:
https://github.com/RAMKUMAR63815/smart-ecommerce-platform

The GitHub repository contains the project source code, Postman files, configuration templates, README, and supporting documentation.

30. CONCLUSION

The Smart E-Commerce Platform demonstrates the implementation of a secure and modular e-commerce backend.
The project combines authentication, authorization, database management, business APIs, testing, documentation, and version control into a complete assignment.